

图片压缩与解压缩方法说明

BSR1901 的 SDK 支持在图片编码的时候，对图片进行压缩，以减少对 Flash 资源的占用。我们的压缩编码采用的是游程编码（Run-Length Encoding, RLE）压缩算法，这种方式的优点是压缩和解压缩都非常简单。

我们有一个专门的工具 LCD_mcuTool.exe（最新的是 lcd_mcu_tool V0.2.4.9.exe，感谢 WeGui 卢工的技术支持），在 PC 上完成压缩的编码，在 BSR1901 芯片里面完成解码（解压缩）。

数据的表达方法（数据格式）是这样的：

在"RGB565 原始 RLE 压缩"数据储存格式 =

```
{
    dat0 的数量,dat0h,dat0l,
    dat1 的数量,dat1h,dat1l,
    ...,
    datn 的数量,datnh,datnl,
    0x00,//0x00 表示结束
};
```

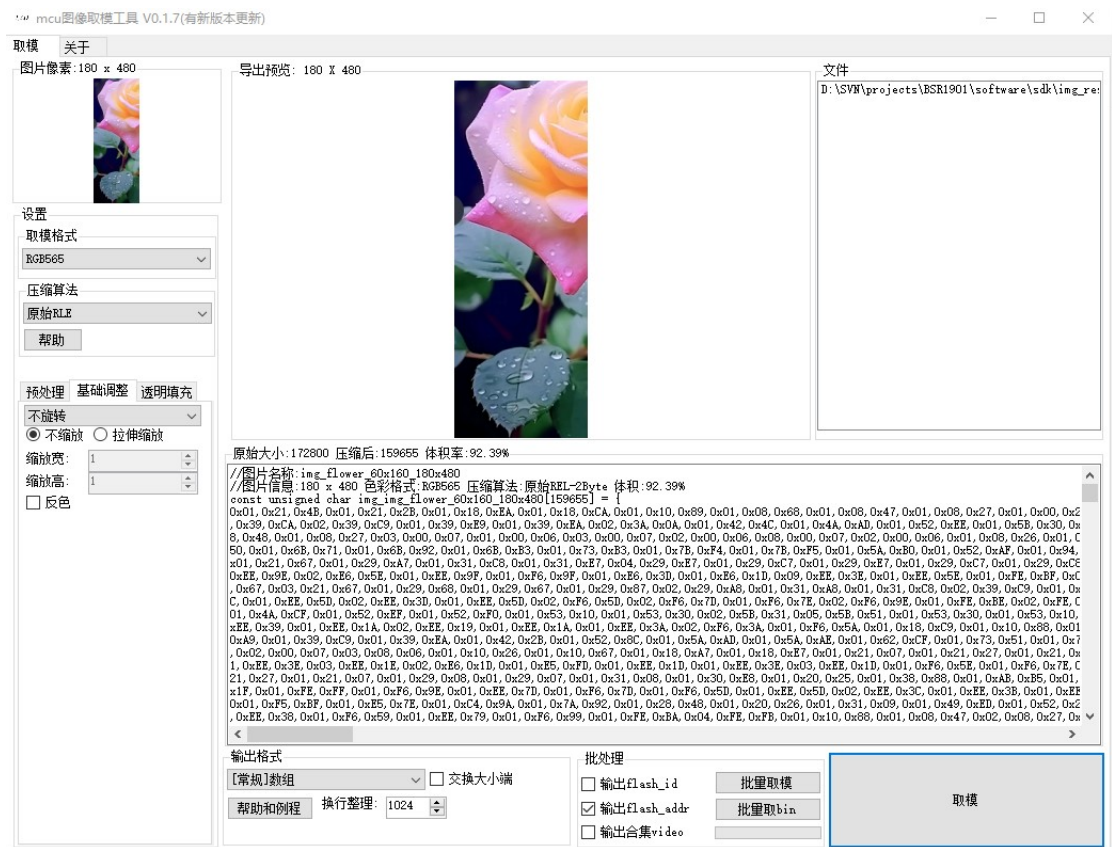
图片 1 示例:

RLE 压缩，体积变成了原始数据的 16.07%，差不多是原来的五分之一。



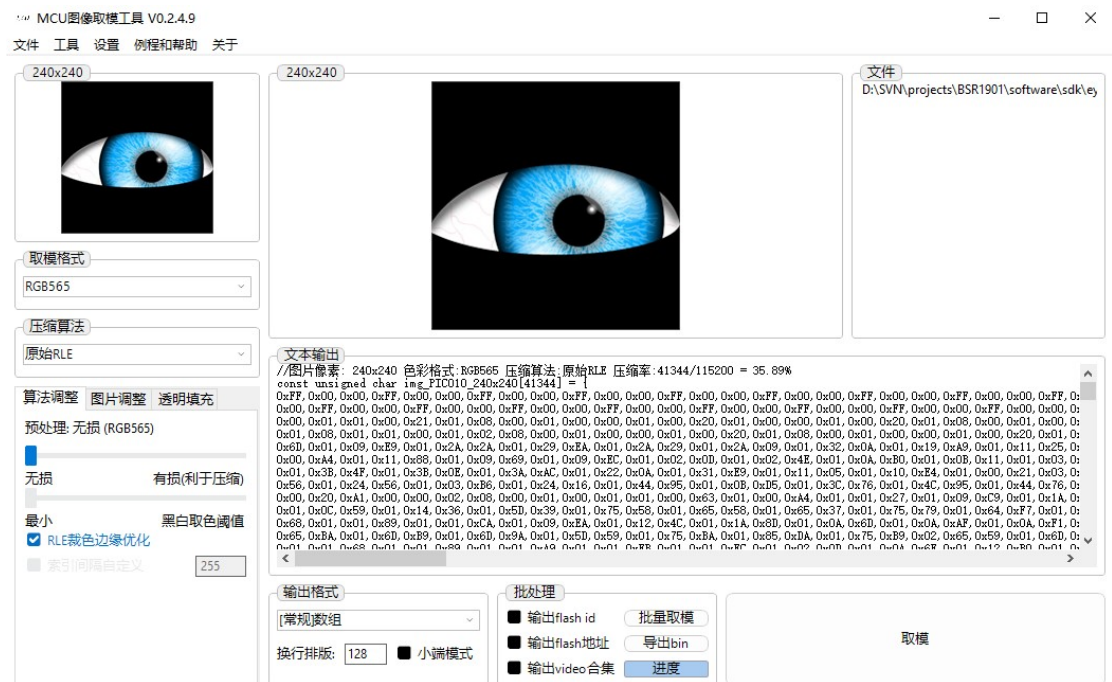
图片 2 示例：

RLE 压缩，体积变成了原始数据的 92.39%，压缩效果变差了。

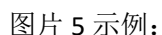


图片 3 示例：

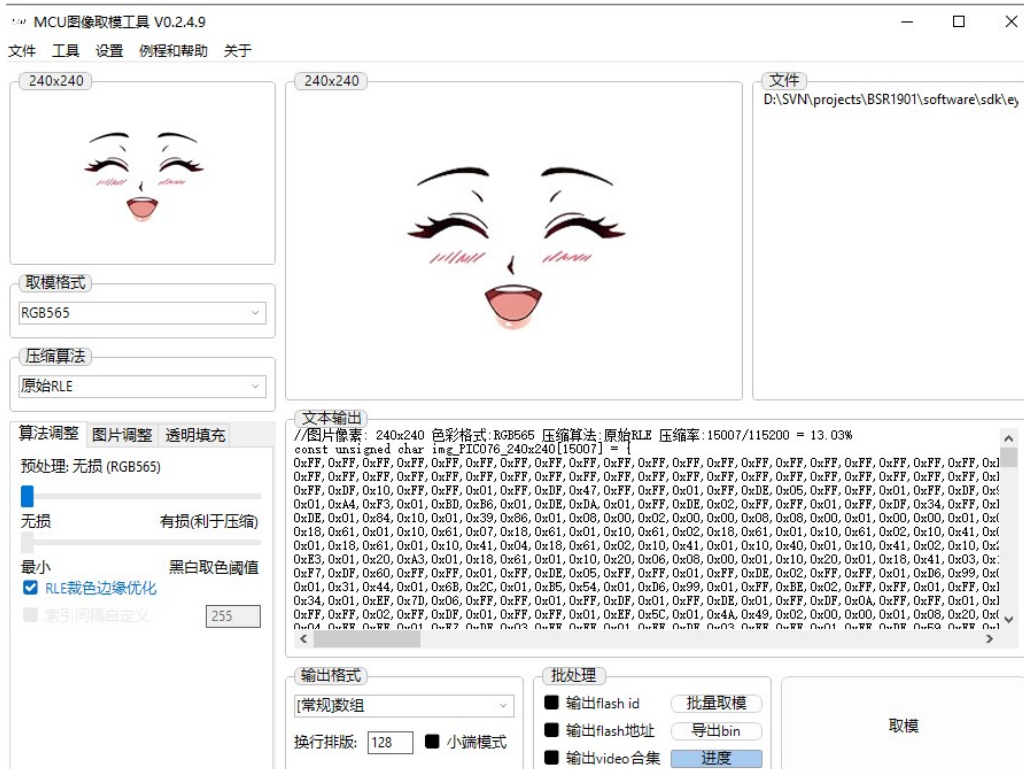
RLE 压缩，体积变成了原始数据的 35.89%，差不多是原来的三分之一。



RLE 压缩，体积变成了原始数据的 59.67%，差不多是原来的二分之一。



RLE 压缩，体积变成了原始数据的 13.03%，差不多是原来的十分之一。



BSR1901 的解压缩方法:

//RGB565 原始 RLE 数据解压例程:

```
void TFT565_draw_arry_rle2_img(uint16_t x, uint16_t y, uint16_t sizex, uint16_t sizey, const uint8_t *arry)
{
    // 设置显示区域（只执行 1 次，不动它）
    Lcd_SetRegion(x, y, x + sizex - 1, y + sizey - 1);
    // 调用 RAM 中的解码函数
    RLE_Decode_Send(arry);
}

// =====
// 配置宏：可自由调整 DMA 触发阈值
// =====
#define RLE_DMA_THRESHOLD 8 // 像素数 > 此值时使用 DMA，否则 CPU 直接发送
__RAM_CODE__ static void RLE_Decode_Send(const uint8_t *arry)
{
    uint8_t count;
    uint8_t dat[2];
    uint16_t color;
    const uint8_t *p = arry;
    for (;;)
    {
        count = *p++;
        if (count == 0) break;
        // 读取 RGB565 颜色
        dat[0] = *p++;
        dat[1] = *p++;
        if (count <= RLE_DMA_THRESHOLD)
        {
            // 短数据：CPU 直接发，比 DMA 快
            do
            {
                SPI_Write16bitData(dat[0]<<8|dat[1]); //LSBF (1<<6) // 1'b0:MSB first (高位在前)
            } while (--count);
        }else
        {
            // 长数据：DMA 批量发送，效率拉满
            color = (uint16_t)dat[0]<<8|dat[1];
            HW_SPI_Tx_DMA_16bit_ColorBlock(&color,count);
        }
    }
}
```

BSR1901 支持 8bit,16bit,24bit,32bit SPI 帧格式,所以一个 16bit(RGB565)像素,BSR1901 一条指令就执行完了。

对于相同的,一样的批量数据, BSR1901 的 DMA,可以支持 SRC_NOINC 模式,也是一条指令就行完了。

这种长数据和短数据分开处理的方法,大大提高了解压缩的速度。

在配置 DMA 的时候, 一个是 **SRC_INC**, 另一个是 **SRC_NOINC**。

```
void HW_SPI_Tx_DMA_16bit(HAL_SPI_ID_T id,uint16 *pData, uint16 DataLen)
{
    LCD_RS_SET;
    SPI_16bit_Transfer();
    Gecko_DMA_Transport((volatile uint32 *)(XR7_SPI_BASE + XR7_SPI_FIFO), pData, DataLen,
        AHB_DMA_CONTROL_HWORD_TR, AHB_DMA_CONTROL_SRC_INC_DES_NOINC);
}

void HW_SPI_Tx_DMA_16bit_ColorBlock(HAL_SPI_ID_T id,uint16 *pData, uint16 DataLen)
{
    LCD_RS_SET;
    SPI_16bit_Transfer();
    Gecko_DMA_Transport((volatile uint32 *)(XR7_SPI_BASE + XR7_SPI_FIFO), pData, DataLen,
        AHB_DMA_CONTROL_HWORD_TR, AHB_DMA_CONTROL_SRC_NOINC_DES_NOINC);
}
```

在做 LCD 显示色块测试的时候, 我建议调用 **HW_SPI_Tx_DMA_16bit_ColorBlock** 函数, 范例如下:

```
#define RED                0xf800
#define GREEN              0x07e0
#define BLUE               0x001f
#define WHITE              0xffff
#define BLACK              0x0000

uint16_t blue_color = BLACK;// RED ;// GREEN;//BLUE;
lcd_dma_refresh_colorblock(0, 0, X_MAX_PIXEL, Y_MAX_PIXEL,&blue_color);

void lcd_dma_refresh_colorblock(uint16_t xs, uint16_t ys, uint16_t w, uint16_t h, color_t *color)
{
    uint32_t len = w*h;
    Lcd_SetRegion(xs, ys, xs+w-1, ys+h-1);
    HW_SPI_Tx_DMA_16bit_ColorBlock(HAL_SPI_0,color,len);
}
```

集成 scgui，Keil 生成的报告显示：

Program Size: Code=18340 RO-data=297108 RW-data=164 ZI-data=2664

SRAM Analysis (3576/4096 bytes = 87.3% used)

Item	字节	占比	文件
g_pfb_buf (帧缓冲)	2400	58.6%	sc_common.c
RAM_CODE (SRAM执行代码)	748	18.3%	xt25f32, dma, isr
sc_task[4] (线程池)	80	2.0%	sc_event_task.c
sc_dirty_buckets[8] (脏区桶)	64	1.6%	sc_common.c
eventQueue (事件队列)	52	1.3%	sc_event_task.c
bms_ip2366 标志+变量	49	1.2%	bms_ip2366.c
font_dsc + lv_font_16	48	1.2%	lv_font_16.c
my_charger (含 PID float)	40	1.0%	charger.c
tft (GUI结构体)	28	0.7%	sc_gui.c
其他 RW-data	~67	1.6%	various

g_pfb_buf 的计算：

#define SC_SCREEN_WIDTH (240)

#define SC_PFB_BUF_SIZE (SC_SCREEN_WIDTH * 5) // = 240 × 5 = 1200

2400 字节 = 240 (屏幕宽度) × 5 (行高) × 2 (每像素 2 字节, RGB565)

如果换成 BSR1901 原生的 Gui 函数（bsr1901_GUI.c），这样 SRAM 占用非常少了，Keil 的报告显示：Program Size: Code=14412 RO-data=292820 RW-data=96 ZI-data=40

Code (代码)	14,412 bytes
RO Data (只读)	292,820 bytes
RW Data (已初始化)	96 bytes
ZI Data (零初始化/BSS)	40 bytes
SRAM 静态占用	136 bytes (3.3%)
RAM_CODE (SRAM执行代码)	740 bytes
SRAM 总计占用	876 bytes (21.4%)
可用 SRAM	4,096 bytes

建议客户根据应用需要选择。